

JDBC 레이어링과 CRUD 구현

오늘의 강의 학습 요약

본 강의는 JDBC의 핵심 특징(데이터베이스/실행 환경 독립성)과 JDBC API가 인터페이스 중심으로 설계된 이유(벤더 드라이버의 구현 책임)를 정리한 뒤, JDBC 연동의 표준 절차(드라이버/URL/계정 정보, 로딩, 커넥션, SQL, PreparedStatement, 실행, 자원 해제, 트랜잭션)를 구조적으로 재정리합니다.

이후 실무형 3계층 구조(Presentation-Service-DAO)로 JDBC 코드를 분리하는 기준(1~7 단계의 배치)을 확립하고, DTO로 레이어 간 데이터를 전달하는 패턴을 적용하여 Insert/Delete/Update/Select를 구현합니다. 또한 PK 중복 및 삭제 대상 없음 같은 상황에서 사용자 친화적 메시지를 제공하기 위해 커스텀 예외를 만들어 예외를 상위 레이어로 위임(throw/throws)하는 패턴까지 완성합니다.

1. JDBC 아키텍처와 핵심 API

JDBC는 드라이버 매니저를 제외한 주요 구성요소가 인터페이스로 제공되며, 벤더 드라이버가 이를 구현하여 어떤 DB/환경에서도 일관된 방식으로 접근할 수 있습니다.

JDBC는 자바 프로그램이 데이터베이스에 접근하기 위한 표준 기술이며, 특정 DB에 종속되지 않도록 설계되어 있습니다. 즉 MySQL, Oracle, MSSQL 등 벤더가 달라도 JDBC API 사용 방식은 동일하며, Java SE 환경뿐 아니라 웹/엔터프라이즈 환경에서도 동일한 연결 방식이 유지됩니다.

JDBC API의 구조적 핵심은 “인터페이스 중심 설계”입니다. `DriverManager` 는 대표적인 클래스이지만, `Connection` , `PreparedStatement` , `ResultSet` 등은 인터페이스로 제공됩니다. 자바 진영은 이 인터페이스(추상 메서드 집합)를 정의하고, 각 DB 벤더는 이를 실제로 동작하도록 구현한 드라이버(JAR)를 제공합니다.

드라이버는 IDE에서 사용 가능하도록 반드시 빌드 패스(의존성)에 포함되어야 합니다. 이 과정은 Eclipse든 IntelliJ든 동일하게 필요하며, “드라이버 다운로드 + 프로젝트 클래스패스 등록”이 JDBC의 사전 조건으로 정리됩니다.

구성 요소	형태	역할 요약
<code>DriverManager</code>	클래스	URL/계정정보로 커넥션을 얻는 진입점 역할을 수행합니다.
<code>Connection</code>	인터페이스	DB 세션/트랜잭션 단위의 연결을 표현하며 commit/rollback/close를 제공합니다.
<code>PreparedStatement</code>	인터페이스	SQL을 전송하고 파라미터 바인딩을 지원하며 executeUpdate/executeQuery를 제공합니다.
<code>ResultSet</code>	인터페이스	SELECT 결과를 커서 기반으로 순회하며 컬럼 값을 타입별 getter로 조회합니다.

핵심 키워드

#JDBC

#인터페이스 기반 API

#벤더 드라이버

#DriverManager

#빌드패스

2. JDBC 연동 7단계와 예외/자원 관리

JDBC 코드는 드라이버/URL/계정 준비부터 로딩, 커넥션, SQL 준비, 실행, 결과 처리, 자원 반납까지 정형화된 단계로 구성되며 모든 과정에서 예외 처리와 close가 필수입니다.

JDBC 연동은 “정해진 절차”가 반복되는 구조이므로, 단계 자체를 명확히 외우고 이해하는 것이 중요합니다. 특히 `Class.forName()` 으로 드라이버 클래스를 메모리에 로딩하는 과정, 그리고 JDBC 메서드 전반이 `SQLException` 계열 예외를 강제하므로 `try-catch` 가 필수라는 점이 강조됩니다.

연동에 필요한 기본 정보는 4가지로 정리됩니다. 드라이버 클래스명은 문자열로 지정되며, URL은 DB 설치 위치 (호스트/포트/DB명 등)를 나타내고, 계정과 비밀번호가 함께 필요합니다.

정보	의미
Driver Class	벤더가 제공한 JDBC 드라이버 클래스의 FQCN 문자열입니다.
URL	DB 접속 위치(로컬/서버, 포트, DB 식별자)를 나타냅니다.
UID	DB 계정(사용자)입니다.
Password	DB 비밀번호입니다.

DML과 SELECT는 실행 메서드와 반환값이 다릅니다. DML은 반영된 행 수를 `int` 로 돌려주며, SELECT는 결과 집합을 `ResultSet` 으로 돌려줍니다. 이 반환값은 “화면에서 사용자에게 무엇을 보여줄지”와 직접 연결되므로, 상위 레이어에 전달하기 위한 설계가 중요합니다.

구분	실행 메서드	반환	의미
DML(INSERT/UPDATE/DELETE)	<code>executeUpdate()</code>	<code>int</code>	반영된 행 수로 성공 여부를 판단합니다.
SELECT	<code>executeQuery()</code>	<code>ResultSet</code>	커서 기반으로 행/열을 순회하며 값을 꺼냅니다.

`ResultSet` 은 테이블 자체가 아니라 “테이블 결과를 객체화한 커서”로 이해하는 것이 핵심입니다. `next()` 는 커서를 다음 행으로 이동시키며, 더 이상 행이 없으면 `false` 가 되어 반복 종료 조건이 됩니다. 컬럼 값은 타입에 따라 `getInt` , `getString` 등으로 꺼내며, 컬럼 지정은 “컬럼 헤더명” 또는 “SELECT 순서 기반 인덱스 (1,2,3...)”가 가능합니다.

컬럼 인덱스 지정은 편하지만 가독성이 떨어질 수 있으므로, 특별한 이유가 없으면 컬럼 헤더명을 사용하는 것이 유
지보수에 유리합니다. 인덱스를 쓰면 SQL의 SELECT 컬럼 구성을 다시 확인해야 매핑이 명확해지는 문제가 생깁
니다.

자원 반납은 반드시 수행되어야 하며, 사용한 순서의 역순으로 `close()` 하는 것이 원칙입니다. SELECT는
`ResultSet` 이 존재하므로 `ResultSet` → `PreparedStatement` → `Connection` 순으로 닫고, DML은
`PreparedStatement` → `Connection` 중심으로 닫습니다.

트랜잭션은 “묶음 처리”이며, JDBC의 기본 동작은 오토커밋입니다. 즉 DML 수행 시 즉시 반영됩니다. 묶음 단위
로 성공/실패를 통제하려면 오토커밋을 끄고(`setAutoCommit(false)`) 성공 시 `commit()` , 실패 시
`rollback()` 으로 처리합니다.

핵심 키워드

`#Class.forName``#SQLException``#executeUpdate``#ResultSet``#트랜잭션`

3. 3계층 구조와 DTO 데이터 전달 패턴

Presentation-Service-DAO로 레이어를 분리하면 책임이 명확해지며, 레이어 간 데이터 전달은 DTO와 컬렉션(List) 조합으로 표준화됩니다.

현업형 구조는 크게 Presentation(사용자와 직접 상호작용), Service(비즈니스 로직 및 트랜잭션/흐름 제어), DAO(Persistence, DB 연동)로 분리됩니다. 혼자 개발하는 프로젝트가 아니라면 역할 분리를 통한 협업/유지보 수성이 필수이므로, 이 구조를 피해가기 어렵다는 점이 강조됩니다.

레이어로 분리하면 "데이터가 레이어 사이를 왕복"해야 하므로 전달 방식이 중요해집니다. 컬럼이 많은 레코드를 매개변수 여러 개로 쪼개 전달하는 것은 비효율적이므로, 레코드 단위를 담는 DTO(Data Transfer Object)를 만들어 한 번에 전달하는 패턴을 사용합니다. 여러 레코드라면 DTO 여러 개를 `List` 같은 컬렉션에 담아 전달합니다.

전달 상황	권장 방식	설명
레코드 1건 전달	DTO 1개	컬럼 값을 DTO 필드에 담아 레이어 간 전달합니다.
레코드 N건 전달	<code>List<DTO></code>	각 행을 DTO로 만든 뒤 컬렉션으로 묶어 전달합니다.

또한 "사용자에게 보이는 메시지 출력"은 Presentation 레이어(메인) 책임으로 고정합니다. DAO에서 "저장 성공" 같은 메시지를 출력하면 레이어 역할이 섞이므로, DAO는 결과값(예: `executeUpdate()` 의 `n`)을 반환하고 메인이 출력 판단을 수행해야 합니다.

핵심 키워드

#3계층 구조

#Presentation

#Service

#DAO

#DTO

4. 레이어 분리 기준: JDBC 1~7단계 배치

JDBC 1~7단계를 레이어에 어떻게 배치하느냐가 분리의 핵심이며, Connection은 Service에서 관리하고 Statement/ResultSet은 DAO에서 관리합니다.

단일 파일로 JDBC를 작성하면 1~7단계를 순서대로 한 곳에서 처리하면 되지만, 레이어를 나누면 각 단계의 책임 소재를 명확히 나뉘어야 합니다. 이때 핵심 기준은 “누가 커넥션을 만들고 닫는가”이며, 트랜잭션 경계가 서비스에 있어야 하므로 커넥션은 서비스에서 얻고 반납하는 구조로 정리됩니다.

DAO는 커넥션을 직접 만들지 않고(서비스로부터 전달받고), SQL 작성/PreparedStatement 생성/실행/ResultSet 처리 및 자신이 만든 자원만 close하는 역할을 가집니다. 즉 DAO는 PreparedStatement 와 ResultSet (SELECT인 경우)만 닫고, Connection.close() 는 하면 안 됩니다.

단계(요약)	Service 레이어	DAO 레이어
1. 접속정보(드라이버/URL/계정)	보관 및 관리합니다.	직접 관리하지 않습니다.
2. 드라이버 로딩 (Class.forName)	생성자 등에서 처리합니다.	처리하지 않습니다.
3. 커넥션 획득	수행하고 DAO에 전달합니다.	전달받아 사용합니다.
4~6. SQL 준비/전송/실행	수행하지 않습니다.	SQL/PreparedStatement/execute를 수행합니다.
7. 자원 해제(close)	Connection만 닫습니다.	Statement/ResultSet만 닫습니다.

서비스를 인터페이스 + 구현체로 분리하는 방식도 함께 사용됩니다. 이는 확장/대체/테스트 용이성 측면에서 일반적인 구조이며, 오버라이딩 규칙(메서드 시그니처/throws 일치)이 이후 예외 위임에서 중요해집니다.

핵심 키워드

#Connection 관리

#DAO 자원 해제

#Service 인터페이스

#레이어 책임 분리

#오버라이딩

5. Insert 구현: DTO 전달과 반환값 위임

Insert는 사용자 입력을 DTO로 묶어 Service→DAO로 전달하고, DAO는 반영 행 수(int)를 반환하여 메인에서 성공 메시지를 결정합니다.

Insert는 DML이므로 핵심 결과는 “몇 건 반영되었는가”입니다. DAO는 `executeUpdate()` 의 반환값 `n` 을 받으며, 이 값이 메인까지 전달되어야 “저장 성공” 같은 사용자 메시지를 출력할 수 있습니다. 따라서 DAO 메서드는 `void` 가 아니라 `int` 를 반환하도록 바꾸고, 서비스 또한 그 반환값을 메인으로 전달합니다.

사용자 입력은 Presentation에서 발생하며, 예시로 부서번호/부서명/부서위치 3개 값을 스캐너로 입력받습니다. 이 3개 값은 DTO에 담겨 서비스에 전달되고, 서비스는 커넥션을 얻어 DAO에 `Connection + DTO` 를 전달합니다.

레이어	핵심 작업
Main(Presentation)	입력을 받고 DTO를 만들며, 반환된 n으로 성공 메시지를 출력합니다.
Service	커넥션을 획득/해제하고 DAO 호출 결과를 상위로 반환합니다.
DAO	SQL 실행 및 <code>PreparedStatement.close()</code> 를 수행하고 n을 반환합니다.

핵심 키워드

#INSERT

#executeUpdate

#반영 행 수(n)

#DTO 전달

#메시지 출력 책임

6. 예외 위임 패턴: PK 중복을 커스텀 예외로 변환

DAO에서 발생한 SQL 예외를 커스텀 예외로 변환해 throw하고, Service는 throws로 위임하여 메인에서 getMessage로 사용자 친화적 메시지를 출력합니다.

PK 중복 삽입 시 DB에서는 무결성 제약 위반 예외(대표적으로

SQLIntegrityConstraintViolationException 계열)가 발생합니다. 이를 DAO에서 printStackTrace() 로 출력하면 사용자가 이해하기 어렵고 레이어 책임에도 맞지 않으므로, DAO는 예외를 “사용자용 메시지를 가진 커스텀 예외”로 바꿔 던지고, 최종 메시지 출력은 메인에서 처리하도록 위임합니다.

커스텀 예외는 Exception 을 상속받아 만들며, 문자열 메시지를 받는 생성자 하나만 두고 super(message) 로 위임하는 형태로 구성합니다. 그리고 DAO의 catch(SQLException e) 등에서 throw new DuplicatedDptNoException(...) 처럼 명시적으로 다시 발생시킵니다.

```
public class DuplicatedDptNoException extends Exception {  
    public DuplicatedDptNoException(String message) {  
        super(message);  
    }  
}
```

이 패턴에서 중요한 포인트는 “catch로 잡아버리면 위임할 예외가 사라진다”는 점입니다. 따라서 DAO에서 SQL 예외를 잡은 뒤, 메인까지 올리기 위한 새 예외를 만들어 throw 해야 합니다. 그 다음 Service와 Service 인터페이스는 오버라이딩 규칙을 만족하도록 동일한 throws 를 선언해야 하며, 마지막으로 메인에서 try-catch 로 잡아 getMessage() 를 출력합니다.

위치	처리 방식	목적
DAO	SQL 예외를 잡고 커스텀 예외로 변환하여 throw 합니다.	기술적 예외를 도메인/사용자 메시지로 전환합니다.
Service(+인터페이스)	메서드 시그니처에 throws 로 위임합니다.	오버라이딩 규칙을 지키며 상위로 전달 합니다.
Main	try-catch 로 잡고 e.getMessage() 를 출력합니다.	사용자 커뮤니케이션을 한 곳에서 통제 합니다.

DAO에 남아있는 printStackTrace() 는 디버깅 목적이므로 사용자 메시지 처리로 전환하는 시점에는 주석 처리하여 “빨간 로그”가 사용자에게 노출되지 않도록 정리합니다.

핵심 키워드

#커스텀 예외

#throw/throws

#오버라이딩 규칙

#getMessage

#PK 중복

7. Delete 구현과 '삭제 대상 없음' 예외(명시적 예외)

Delete는 DML이므로 `n==0`을 조건으로 '존재하지 않는 레코드' 예외를 명시적으로 발생시키고, 동일한 위임 패턴으로 메인에서 메시지를 출력합니다.

Delete는 부서번호(DPTN) 하나로 삭제하는 구조로 단순화되며, DTO 대신 `int dptNo` 같은 형태로 전달해도 됩니다. SQL 관점에서는 존재하지 않는 키로 삭제해도 예외가 발생하지 않고 단지 `n==0` 이 반환되므로, "시스템 예외는 아니지만 업무적으로 예외"인 상황을 프로그램이 정의해야 합니다.

이를 위해 `RecordNotFoundException` 같은 예외를 별도로 만들고, DAO에서 `n==0` 일 때 `throw new RecordNotFoundException("...")` 로 명시적으로 발생시킵니다. 이후 Service→Main으로 동일하게 위임하여 메인에서 사용자 메시지를 출력합니다.

```
public class RecordNotFoundException extends Exception {  
    public RecordNotFoundException(String message) {  
        super(message);  
    }  
}
```

상황	DB 동작	애플리케이션 처리
존재하는 부서번호 삭제	n이 1 이상으로 반환됩니다.	메인에서 "삭제 성공"을 출력합니다.
존재하지 않는 부서번호 삭제	n이 0으로 반환됩니다.	커스텀 예외를 발생시켜 "존재하지 않음"을 안내합니다.

핵심 키워드

#DELETE

#n==0 처리

#RecordNotFoundException

#명시적 예외

#업무 예외

8. Update 구현: DML 패턴 재사용과 DTO 입력 설계

Update는 Insert와 동일한 DML 골격을 재사용하며, 수정 대상 번호와 변경 값들을 DTO로 묶어 전달하는 설계를 적용합니다.

Update도 DML이므로 DAO/Service 구조는 Insert와 동일하게 재사용됩니다. 강의에서는 “수정할 부서번호 + 수정할 부서명 + 수정할 위치”를 입력으로 받아 DTO에 담아 전달하는 방식으로 구현합니다. 따라서 DAO의 파라미터는 DTO가 되고, SQL의 바인딩 순서에 맞춰 `setString`, `setInt` 등을 적용합니다.

Insert/Delete/Update는 모두 “SQL만 달라지고 뼈대는 동일”하므로, 하나를 정확히 정리해 두면 나머지 DML은 빠르게 확장할 수 있습니다. 또한 복사/붙여넣기로 기능을 합칠 때는 패키지/임포트가 이전 프로젝트를 참조하지 않도록 반드시 상단 import를 점검해야 합니다.

핵심 키워드

#UPDATE

#DML 공통 뼈대

#DTO 바인딩

#복사 후 import 점검

#재사용

9. 최종 통합: CRUD를 하나의 메뉴형 프로그램으로 결합

개별 프로젝트로 구현했던 Select/Insert/Update/Delete를 하나의 Final 프로젝트로 통합하고, 메인에서 메뉴 기반으로 기능을 호출하도록 구성합니다.

마지막에는 Select 중심 프로젝트를 복사해 Final 프로젝트를 만들고, DAO/Service/ServiceImpl에 CRUD 메서드를 모두 모아 “하나의 애플리케이션” 형태로 통합합니다. 이 과정에서 가장 빈번한 문제는 “복사한 코드가 다른 패키지의 클래스를 import하는 것”이므로, 잘못된 import를 삭제하고 Final 프로젝트 패키지로 맞추는 작업이 필수입니다.

또한 Insert/Delete에서 사용했던 커스텀 예외 클래스(`Duplicated...` , `RecordNotFound...`)는 Final 프로젝트에도 존재해야 하므로, 누락되면 클래스를 복사해 포함시켜야 컴파일이 정상화됩니다. 기능 통합은 결국 “메서드 복붙 + import/패키지 정리 + 필요한 예외 클래스 포함”으로 완성됩니다.

메인은 `Scanner` 와 `while(true)` 를 사용해 메뉴를 반복 출력하고, 사용자가 선택한 번호에 따라 CRUD 기능을 실행합니다. 0번 입력 시 스캐너 close 및 프로그램 종료를 수행하며, 목록 출력은 `printf` 를 이용해 컬럼을 보기 좋게 보여주는 형태로 정리됩니다.

메뉴 예시	동작
1	부서 목록을 조회하여 표 형태로 출력합니다.
2	부서 정보를 입력받아 DTO로 저장 요청을 수행합니다.
3	부서번호를 입력받아 삭제를 수행하고 예외 메시지를 처리합니다.
4	부서 정보를 입력받아 수정을 수행합니다.

핵심 키워드

#CRUD 통합

#메뉴 기반 메인

#import 정리

#커스텀 예외 포함

#while(true)